

APPLICATION FOR APPLYING CORRECTORS WITH COUPLED ERROR TO TOLERANCE ANALYSIS AND ERROR BUDGETS

Tyler Seawright
M.S. Mechanical Engineering

ERROR BUDGET OVERVIEW

Error budgets are the backbone of every precision system. First principles work as long as when combined with variance in the inputs, the system performance is within the bounds defined by the error budget. Error budgets enable combination of complex systems and lead to prediction of output behavior.

Engineers design incredibly complex systems and with each new system, a new error budget must be created. There is an ocean of details required to model basic cases and the old method of using Excel spreadsheets is tedious to set up, unstable, and completely inflexible. There is also no clear method to capture coupled error motion from applying an error correction, making 3D tolerance stacks unmanageable. Few software exist, which are difficult to use and costly.

ERROR BUDGET DEFINITION

Error budgets apportion a total error to identify sources to drive design decisions. A good error budget identifies the sources the system is most sensitive to. The error model is more generalized when coupled motion error is considered in 3D space versus linear 1D tolerance stacks. The concept of an error budget stems from GUM [1], serving as a guide for expressing measurement uncertainties “from the shop floor to research instruments.” In this paper, vector means a 1x6 element set of positions and orientations to fully describe a coordinate system in 6 DOF.

ABSTRACT PURPOSE

This abstract is intended to serve as a description of a planned open-source 3D mechanical tolerance stack application that includes a workflow to model complex systems with minimal setup time. A set of MATLAB functions are provided to allow for custom analysis; however, the focus of this abstract will be a single button application to solve 3D tolerance stacks with error correctors running a pre-compiled MATLAB script that reads from actively selected data in Excel. A sample application will be presented in this abstract, highlighting the capabilities of the application and describing the workflow. The application, *TolStack*, UI is shown in Figure 1 [2].

The coordinate transform methods are modeled through multiplying 4x4 element homogeneous transform matrices (HTM) described by [3]. A new method for capturing coupled errors propagated from utilizing correctors is presented to generalize tolerance stacks including error correctors without small angle approximations. A built-in Monte Carlo simulation determines the $\mu + N\sigma$ error representing the mean, μ , and desired output number of standard deviations, $N\sigma$. This application does not focus on distinguishing types of error sources other than mechanical, but they are easily included if a sensitivity is provided. The equations used to convert HTM to positions and angles are from [4].

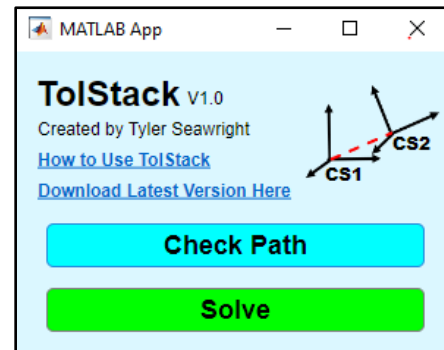


FIGURE 1. *TolStack* V1.0 app user interface.

ERROR MODEL

ERROR TRANSFORM PATH USING HTM

The relative error between two points in six degrees of freedom is calculated by performing an inverse matrix multiplication of the nominal and errored absolute HTMs representing 4x4 matrix transform paths between two coordinate systems. The body transformation is generated by multiplying four base transforms starting with rotations about the z, y, and x axis; then a single position transform by x, y, and z coordinates. Then the coordinate system transforms are assembled and combined. First, the nominal transform path is solved for n coordinate systems in the base coordinate system, using right to left multiplication using eq. 1. Second, the absolute transform path containing propagated errors is solved with an additional error term pre-multiplied with each respective coordinate system described by eq. 2. Lastly, the relative error

transformation between nominal and absolute transform paths is solved by eq. 3.

$$T_0^n = T_0^1 * T_1^2 * ... * T_{n-1}^n \quad (\text{eq. 1})$$

$$Te_0^n = E_0^1 T_0^1 * E_1^2 T_1^2 * ... * E_{n-1}^n T_{n-1}^n \quad (\text{eq. 2})$$

$$Re_0^n = inv(T_0^n) * Te_0^n \quad (\text{eq. 3})$$

CORRECTOR ERROR PROPAGATION

A corrector is a device that corrects an error in a system with respect to a coordinate system. Correctors pose a challenge in precision engineering because when they are used, there may be coupled motion, which is not trivial to quantify for random system error or in 6DOF. The coupled motion to move the system from an arbitrary position transformed to the corrected position is applied. An example of this phenomenon is using a tip/tilt correction, if the rotation axis is not through the system point of interest, a parasitic error motion is added to the system when the correction is performed on one axis. This is most useful when determining the range of adjustments required and if more adjustments are needed. Additionally, this information will indicate if using the corrector pushes a coupled axis out of the required range when used. The order correctors are applied changes the outcome unless using an iterative correction process. Multiple correctors may be defined in the order that they are applied while default is 321 rotations then x-y-z translation if none is specified.

The corrector is applied by solving the corrector difference matrix, which is the HTM representing how far the point of interest is from the corrector on the axes of compensation. The inverse corrector difference transform is then applied the corrector vector to point of interest and then to the absolute position HTM to rotate to the corrector position and propagate the parasitic rotation to the coupled axes.

1. Solve relative HTM error vector E_{re} from the transform using method described by [3].
2. Generate corrector difference vector for all corrector values defined. Ignoring non-corrector defined values is done in program.
 $C_D = C - E_{re} \quad (\text{eq. 4})$
3. Generate transformation from point of interest to corrector T_{VC} .
4. Generate corrector difference transform T_{CD} .
5. Solve absolute errors by applying corrector at a distance from point of interest.

$$T_{cae} = T_{ae}(inv(T_{VC}) * (T_{CD} * T_{VC})) \quad (\text{eq. 5})$$

6. Solve relative error post-applying corrector using method from [4].

MODEL VALIDATION

The model was validated by testing a few cases in which an error was applied to a vector and then corrected.

In the second example, a unit vector in the x direction is used as the transform path. A 17mrad [1°] error is applied about the z axis. The corrector corrects at a resolution of 2.5mrad about the z axis and is located at origin. R_a represents absolute position, R_e represents relative position, and Rec represents corrected relative position.

The results are as expected in Table 1. The corrected error leaves the corrector resolution on the axis of adjustment and the coupled error in x and y was reduced by the delta to move the system to the corrected location. R_{cae} as shown in Figure 2 shows the absolute position after applying the corrector. Table 1 indicates the model outputs of relative displacements with respect to R_a .

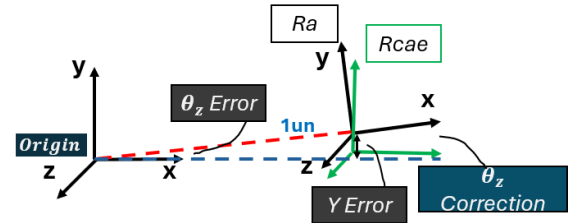


FIGURE 2. An angle error propagated by 1 unit is corrected by a corrector located at the origin.

Vector Name	X	Y	Z	θ_x	θ_y	θ_z
Ra	0.9999	0.0170	0	0	0	0.0170
Re	-0.0001	0.0170	0	0	0	0.0170
Rec	0	0.0025	0	0	0	0.0025

TABLE 1. A 17mrad θ_z rotation error is corrected to 2.5mrad about the origin.

The second error example is made slightly more complicated by moving the corrector position to 2 units on the x axis drawn in Figure 3. Since the corrector is not rotating about the axis through the point of interest, coupled error motion is added when using the corrector. At position $x = 1$ unit from the point of interest, the corrector rotates the

point of interest in the negative direction about z, thus increasing the y position and decreasing the x position with respect to the origin by approximately the same amount. The expected position change between Re and Rec is the difference of angle error and corrector resolution applied as a rotation to the nominal position vector tabulated in Table 3.

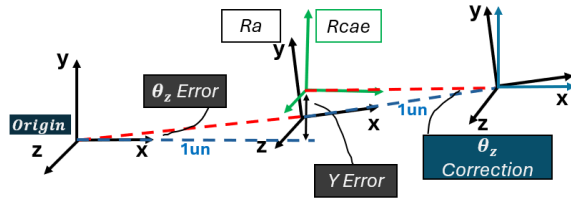


FIGURE 3. An angle error is corrected by a corrector positioned at a place other than the origin, resulting in parasitic position error via angle correction.

Vector Name	X	Y	Z	θ_x	θ_y	θ_z
Ra	0.9999	0.0170	0	0	0	0.0170
Re	-0.0001	0.0170	0	0	0	0.0170
Rec	-0.0004	0.031	0	0	0	0.0025

TABLE 3. Corrected transform results with off-axis rotation.

This is expandable to 6DOF and stacks of any length. With a well-defined math model, an entire mechanical tolerance stack with correctors in different places can be rapidly modeled.

ERROR BUDGET WORKFLOW

DEVELOP MATH MODEL

The math model represents the connection between parts of the error budget. This may include multiple coordinate transformations that are stacked, definition of correctors, and sensitivities within the system. In Excel, the math model is the user's model connecting the stacks performed by the program. For a single stack, no math model is required. System level analysis requires combining stacks.

INPUT VECTOR LOOPS, ERROR TERMS, AND CORRECTORS

In a screenshot, a sketch of the vector path to each part interface is helpful to organize the model. FIGURE 4 depicts the Excel spreadsheet representing the inputs that correspond to the transformation path.

The model is input into a spreadsheet as a list of 6 DOF vectors and relative errors. Since errors are relative, they may be input as the part tolerances in the part's local coordinate system. Each type of input is tagged to make the spreadsheet dynamic for MATLAB to parse as it becomes complex. When the system is solved, the cells containing the inputs and output cells must be highlighted. No other data is considered.

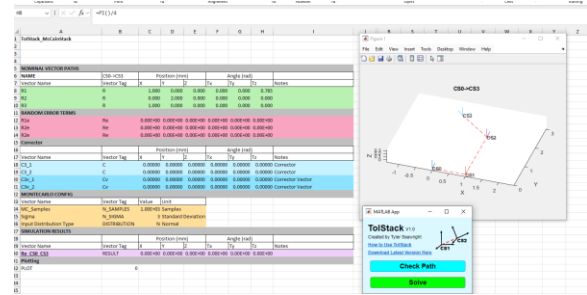


FIGURE 4. The spreadsheet includes all the inputs read by the program. The Error Summary is an example of using a custom math model calculated from the RESULT where the script outputs the solution.

A corrector must be defined with a vector from the corrector to a point of interest and must contain a vector describing the resolution of the corrector per axis of correction. Cells with zeros will be ignored.

MONTECARLO CONFIGURATION

The analysis runs a Monte Carlo requiring input for number of samples and input standard deviation range. By default, the inputs are varied by normal distribution centered at zero.

RUNNING THE MODEL

To perform the analysis, highlight the group of cells containing the tagged inputs and run the macro. The macro will only read and write within the range of highlighted cells such that complex systems with loose formatting can be applied within Excel and only the specified calculations are performed.

REVIEW RESULTS

Results are output to the spreadsheet where specified by tag in the highlighted range. The result is the 6DOF $\mu + N\sigma$ transform vector considering the entire tolerance path where μ is the mean, N is the specified number of standard deviations, and σ is one standard deviation. The script saves results, plots histograms and coordinate transforms depicted in FIGURE 5, and

places figures in the spreadsheet below RESULTS.

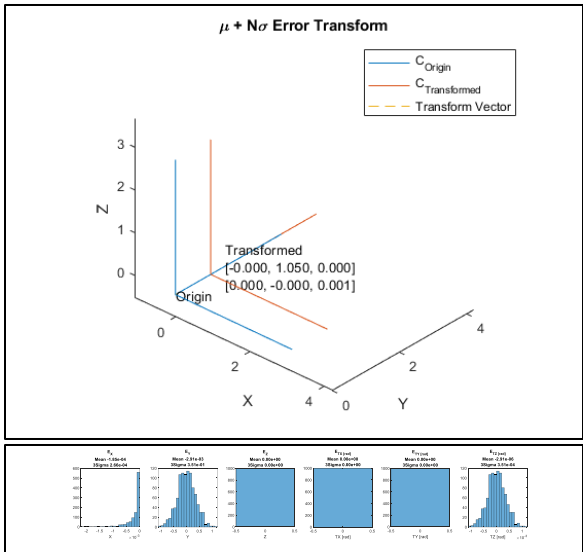


FIGURE 5. A 1m x-axis lever arm wiggled by $\pm 1\text{mrad}$ in theta z for 1000 normally distributed samples results in $\pm 1\text{mm}$ 3σ error in y, $\pm 1\text{mrad}$ 3σ error in theta z, and all x error is negative.

For building larger systems, an additional button and MATLAB function are provided to plot sets of transforms to visualize complex behavior shown in FIGURE 6. Multiple sets of inputs may be combined in one spreadsheet and linked to a master sheet with system level errors making this program scalable. The Check Path button plots the input nominal path.

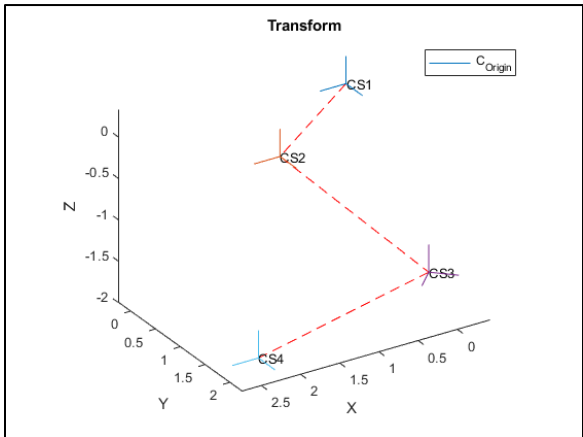


FIGURE 6. A chain of coordinate systems is plotted to represent a system.

SAMPLE APPLICATION: OPTICAL ALIGNMENT

SYSTEM OVERVIEW AND REQUIREMENTS

A sample system is a spherical mirror pair for creating a focusing lens used in a fictional home telescope illustrated by Figure 7. Table 5 describes the mechanical requirements for optic centers with respect to the mechanical subsystem datums. The mirrors are bonded and may be aligned before bonding if needed. The subsystem is mounted by the outer cylinder, datum A, and face, datum B. Since the subsystem is axially symmetric, there is no C datum.

Requirement	Value	Unit
First and Second Mirror, and Pinhole Centration wrt mechanical datums (x,y,z)	0.25 0.25 0.08	mm mm mm
First and Second Mirror Orientation wrt mechanical datums ($\theta_x, \theta_y, \theta_z$)	1.74 1.74 25	mrad mrad mrad

TABLE 4. Telescope lens requirement table. All requirements are in the target coordinate system relative to the tooling target.

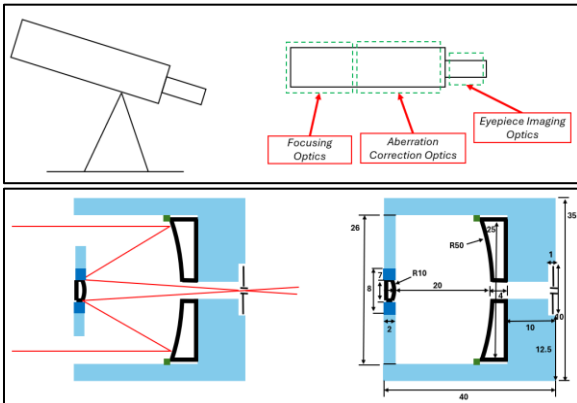


Figure 7. Telescope subsystem, ray trace, and basic dimensions.

ERROR MODEL

The error model is a set of three transform paths from the subsystem origin CS0 defined by mechanical datums as illustrated by Figure 8. Each path is input into Excel and solved independently.

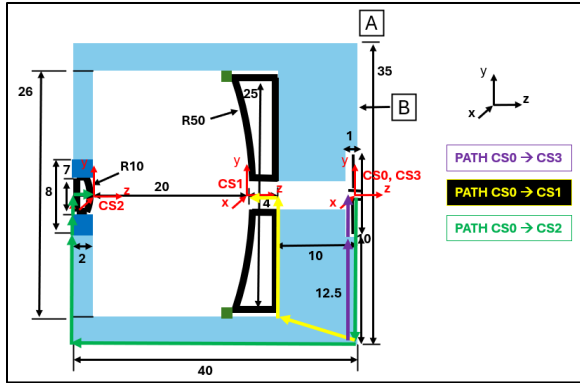


Figure 8. Mechanical datums and transformation path for the telescope objective.

MODEL OUTPUTS

The model outputs the 6DOF relative error histogram plots. Since all optics are fixed by mechanical tolerances, the requirements are not met, and correctors must be used as reported by Figure 9.

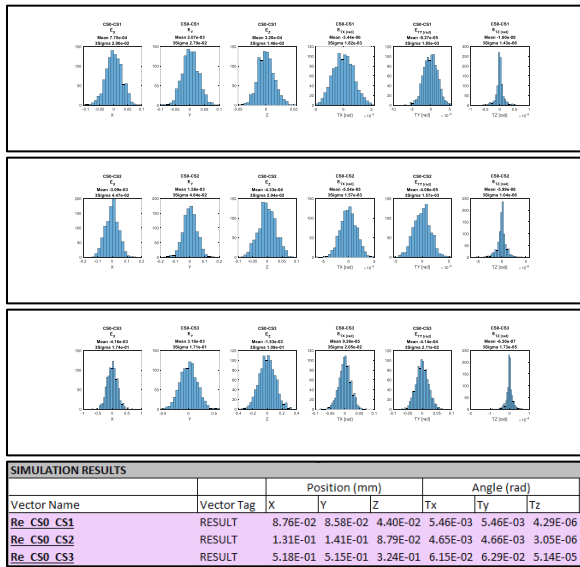


Figure 9. Fixed optic uncertainties do not meet the project requirements.

Aligning the optics before bonding corrects the error enabling the requirements to be met as shown in the corrected model outputs via Figure 10. The distributions for the aligned optics are much tighter due to correctors reducing the standard deviation to the accuracy of the corrector.

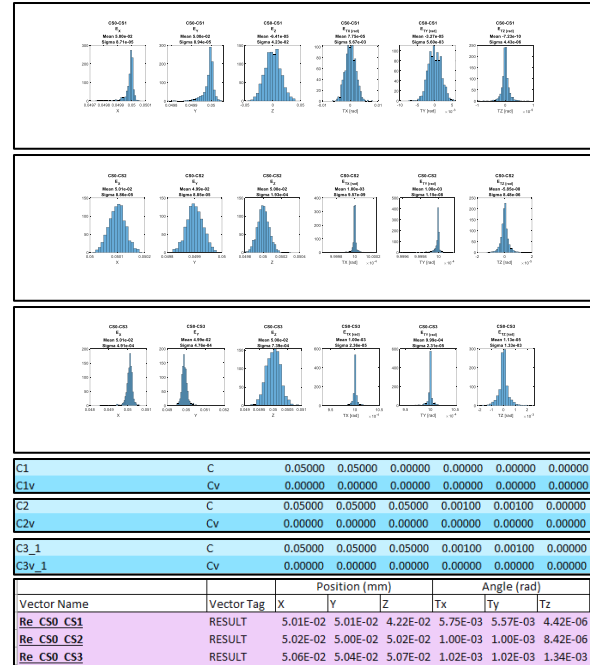


Figure 10. Using correctors to align the second mirror and pinhole enables the mirror and pinhole alignment to meet the requirements.

CONCLUSION

The script is a powerful addition to a mechanical engineer's toolbox enabling rapid tolerance analysis for mechanical tolerance stacks or system level error budgets modeled by HTM error propagation. A new addition to error correction is included to enable modeling use of error correction devices and parasitic errors they impose on the system. This script and macro installation instructions are planned to be entirely open-source and will be made available on GitHub. Demo files will be open sourced with the rest of the script.

FUTURE WORK

This script is modular and may be used in complex analyses and studies for a variety of purposes. Given sensitivities, this script can be used to expand beyond mechanical tolerance stacks if HTM propagation can be used to model the system.

When the program is open sourced, a few functions are provided to build custom programs that quickly generate HTM's or solve the positions and rotations from a given HTM with very few inputs.

This code will be used to model more complex systems in the future and has the potential to aid

in organizing massive systems given the math is performed outside of Excel and Excel is used as the UI. Additional options may be added in the future so that the software can parse such as specific configuration tags or changing input error distribution types.

REFERENCES

- [1] 2025 Joint Committee for Guides in Metrology. Evaluation of measurement data – Guide to the expression of uncertainty in measurement. 2008.
- [2] T. A. Seawright, TolStack V1.0 [Computer Software]. 2025.
<https://github.com/TylerSeawright/TolStack>
- [3] A. H. Slocum, Precision Machine Design. Prentice-Hall, Inc., 1992.
- [4] Duenner, Andrew C. An Error Budgeting Framework for Engineering Design. Massachusetts Institute of Technology. 2019.